

Introduction to Jenkins Pipeline and Triggers

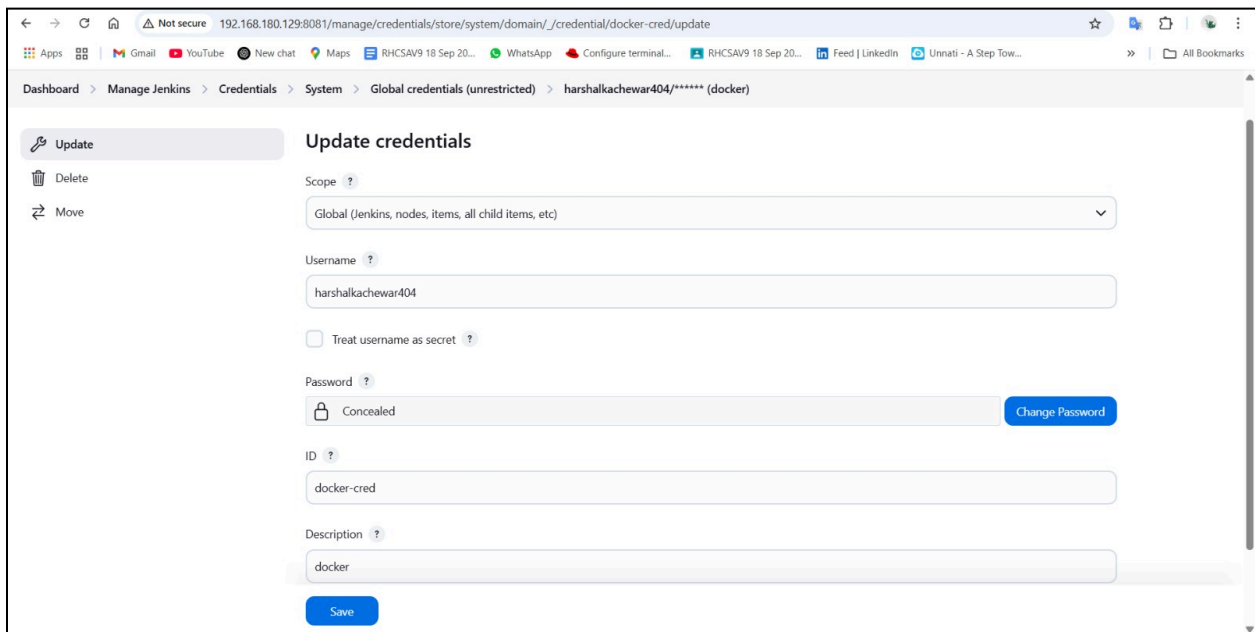
Jenkins Pipeline is a powerful automation tool that allows developers to define their build, test, and deployment workflows using a code-based approach. Pipelines can be triggered based on various conditions, such as code commits, scheduled times, or after another job completes.

Triggering a Build in Jenkins

Triggering a build after another project completes is useful when you need to execute dependent jobs in a sequence. Jenkins provides an option called **"Build after other projects are built"**, which allows a pipeline to start automatically after the successful execution of another.

Step-by-Step Implementation

Step 1: Creating Docker Credentials in Jenkins



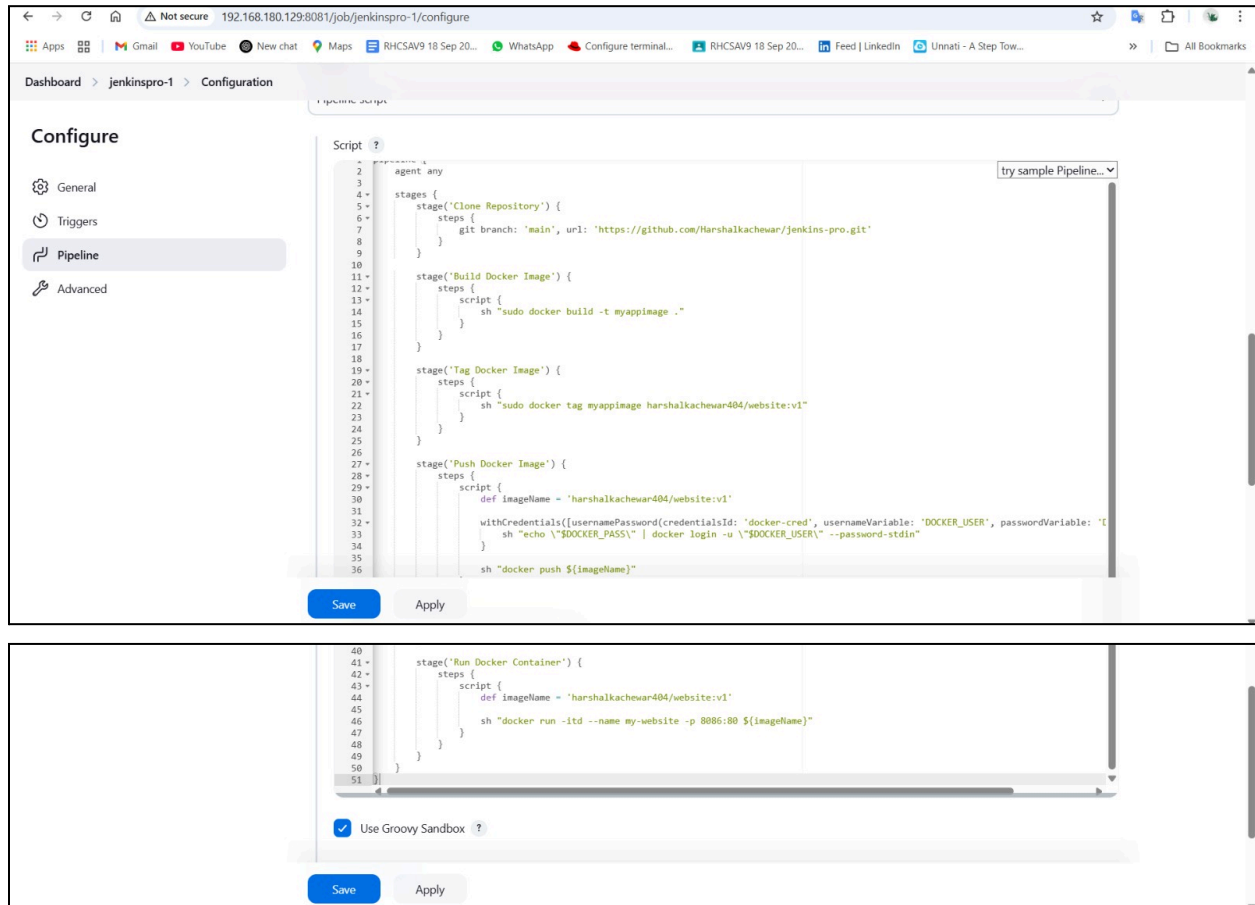
The screenshot shows the Jenkins web interface for updating a credential. The breadcrumb trail is: Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) > harshalkachewar404/***** (docker). On the left, there are buttons for 'Update', 'Delete', and 'Move'. The main form is titled 'Update credentials' and contains the following fields: 'Scope' (a dropdown menu set to 'Global (Jenkins, nodes, items, all child items, etc)'), 'Username' (a text field containing 'harshalkachewar404'), 'Treat username as secret' (an unchecked checkbox), 'Password' (a field with a lock icon and the text 'Concealed', with a 'Change Password' button to its right), 'ID' (a text field containing 'docker-cred'), and 'Description' (a text field containing 'docker'). A 'Save' button is at the bottom left of the form.

Open Jenkins, navigate to Manage Jenkins > Manage Credentials, select Global Credentials, click Add Credentials, choose Username and Password, enter Docker Hub

username and password, set the ID as **docker-cred**, and click OK to save.

Step 2: Creating the First Pipeline (jenkinspro-1)

Open Jenkins, click New Item, enter jenkinspro-1 as the name, select Pipeline, click OK, and in the pipeline script section, enter the following pipeline code:



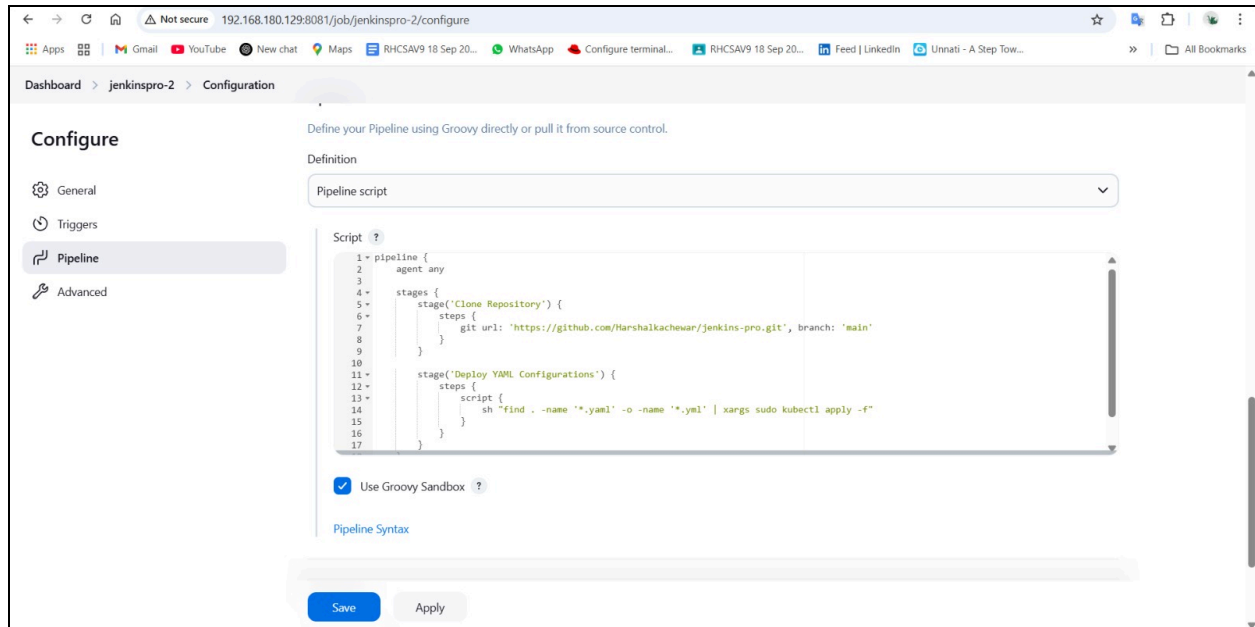
The screenshot shows the Jenkins Pipeline Configuration page for a job named 'jenkinspro-1'. The 'Script' tab is selected, and the pipeline script is entered in the text area. The script defines four stages: 'Clone Repository', 'Build Docker Image', 'Tag Docker Image', and 'Push Docker Image'. The 'Push Docker Image' stage uses a 'docker-cred' credential to push the image to a Docker registry. The 'Run Docker Container' stage is partially visible at the bottom of the script.

```
1 agent any
2
3
4 stages {
5   stage('Clone Repository') {
6     steps {
7       git branch: 'main', url: 'https://github.com/Harshalkachewar/jenkins-pro.git'
8     }
9   }
10
11   stage('Build Docker Image') {
12     steps {
13       script {
14         sh "sudo docker build -t myappimage ."
15       }
16     }
17   }
18
19   stage('Tag Docker Image') {
20     steps {
21       script {
22         sh "sudo docker tag myappimage harshalkachewar404/website:v1"
23       }
24     }
25   }
26
27   stage('Push Docker Image') {
28     steps {
29       script {
30         def imageName = 'harshalkachewar404/website:v1'
31         withCredentials([usernamePassword(credentialsId: 'docker-cred', usernameVariable: 'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) {
32           sh "echo \"\$DOCKER_PASS\" | docker login -u \"\$DOCKER_USER\" --password-stdin"
33         }
34         sh "docker push \$imageName"
35       }
36     }
37   }
38 }
39
40
41 stage('Run Docker Container') {
42   steps {
43     script {
44       def imageName = 'harshalkachewar404/website:v1'
45       sh "docker run -itd --name my-website -p 8086:80 \$imageName"
46     }
47   }
48 }
49
50
51 }
```

At the bottom of the configuration page, the 'Use Groovy Sandbox' checkbox is checked.

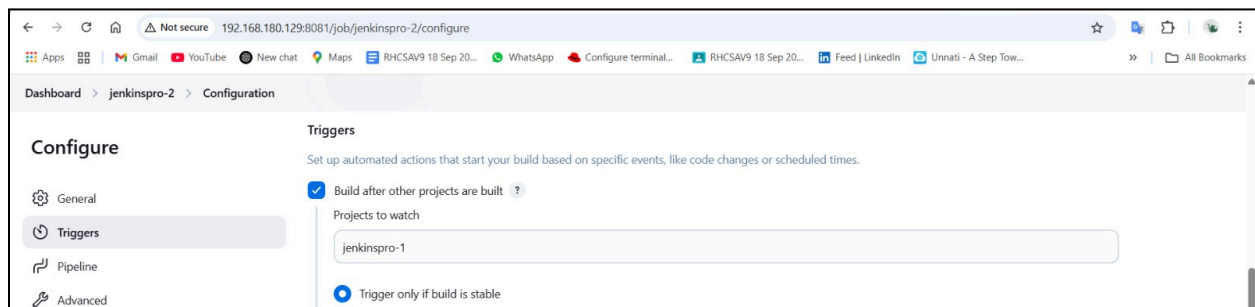
Step 3: Creating the Second Pipeline (jenkinspro-2)

Open Jenkins, click New Item, enter jenkinspro-2 as the name, select Pipeline, click OK, and in the pipeline script section, enter the following code:



Step 4: Setting Up Trigger for jenkinspro-2

Open jenkinspro-2 in Jenkins, click Configure, scroll to Build Triggers, check Build after other projects are built, enter jenkinspro-1, and click Save.



Step 5: Preparing GitHub Repository

Ensure that GitHub repository contains application source code (**myapp**), a Dockerfile (for building the image), and YAML configuration files (for deployment in jenkinspro-2 pipeline).

Step 6: Running jenkinspro-1 and Verifying Automatic Triggering of jenkinspro-2

```
← → ↻ 🔒 Not secure 192.168.180.129:8081/job/jenkinspro-1/1/console ☆ 📄 📁 📌
📱 Apps 📧 Gmail 📺 YouTube 🗺️ Maps 📅 RHCSAV9 18 Sep 20... 📞 WhatsApp 🖱️ Configure terminal... 📅 RHCSAV9 18 Sep 20... 🌐 Feed | LinkedIn 📺 Unnati - A Step Tow...
Dashboard > jenkinspro-1 > #1
8f3c313eb124: Mounted from harshalkachewar404/naruto
b31dcd6d654a: Pushed
c1761f3c364a: Mounted from harshalkachewar404/naruto
08000c18d16d: Mounted from harshalkachewar404/naruto
v1: digest: sha256:c826530fbf18ab9bf9522c65621c97d2aecb04819c4fca2342c2bc94b60341b size: 2198
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Docker Container)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ docker run -itd --name my-website -p 8086:80 harshalkachewar404/website:v1
42f437fdf6290bda2ae9e4d8b557ea879ab9cd05fa0fcf35099feaa2f3f0292
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Triggering a new build of jenkinspro-2 #1
Finished: SUCCESS
```

← → ↻ 🔒 Not secure 192.168.180.129:8081/job/jenkinspro-2/1/console ☆ 📄 📁 📌

📱 Apps 📧 Gmail 📺 YouTube 🗺️ Maps 📅 RHCSAV9 18 Sep 20... 📞 WhatsApp 🖱️ Configure terminal... 📅 RHCSAV9 18 Sep 20... 🌐 Feed | LinkedIn 📺 Unnati - A Step Tow...

Dashboard > jenkinspro-2 > #1

- Status
- <> Changes
- Console Output**
- Edit Build Information
- Delete build '#1'
- Timings
- Git Build Data
- Pipeline Overview
- Pipeline Console
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces

🟢 Console Output

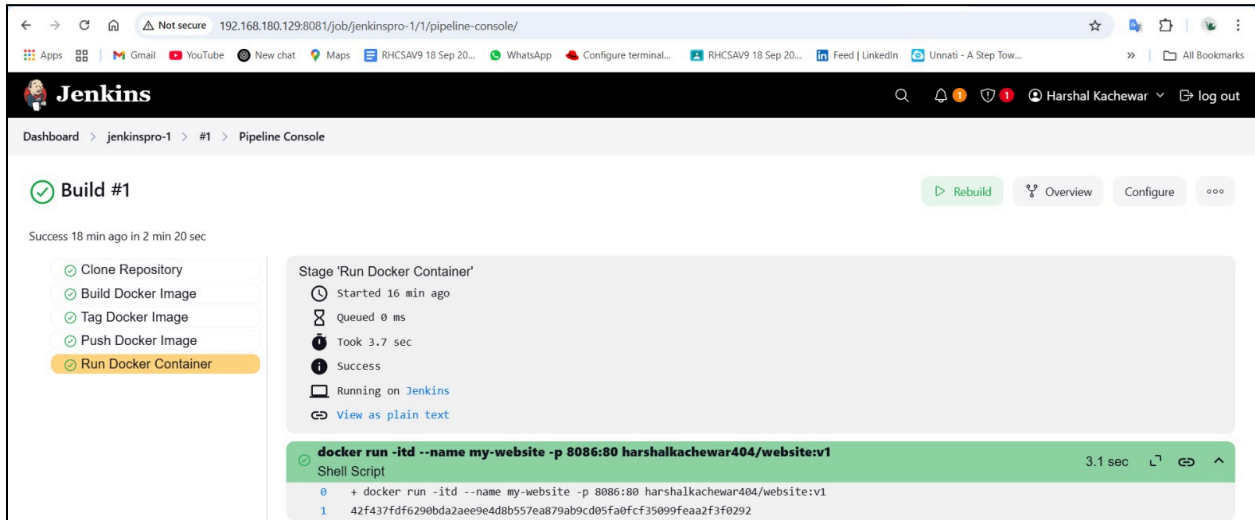
Download Copy View as plain text

Started by upstream project "jenkinspro-1" build number 1
originally caused by:
Started by user Harshal Kachewar

[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/jenkinspro-2

[Pipeline] {
[Pipeline] stage
[Pipeline] { (Clone Repository)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository <https://github.com/Harshalkachewar/jenkins-pro.git>
> git init /var/lib/jenkins/workspace/jenkinspro-2 # timeout=10
Fetching upstream changes from <https://github.com/Harshalkachewar/jenkins-pro.git>
> git --version # timeout=10
> git --version # 'git version 2.34.1'
> git fetch --tags --force --progress -- <https://github.com/Harshalkachewar/jenkins-pro.git> +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url <https://github.com/Harshalkachewar/jenkins-pro.git> # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision cde6cf0027732c0b0f8c7e3daff06727cdea3d4 (refs/remotes/origin/main)

Step 7: Understanding Pipeline Stages for Both Projects



Dashboard > jenkinspro-1 > #1 > Pipeline Console

Build #1 Rebuild Overview Configure ...

Success 18 min ago in 2 min 20 sec

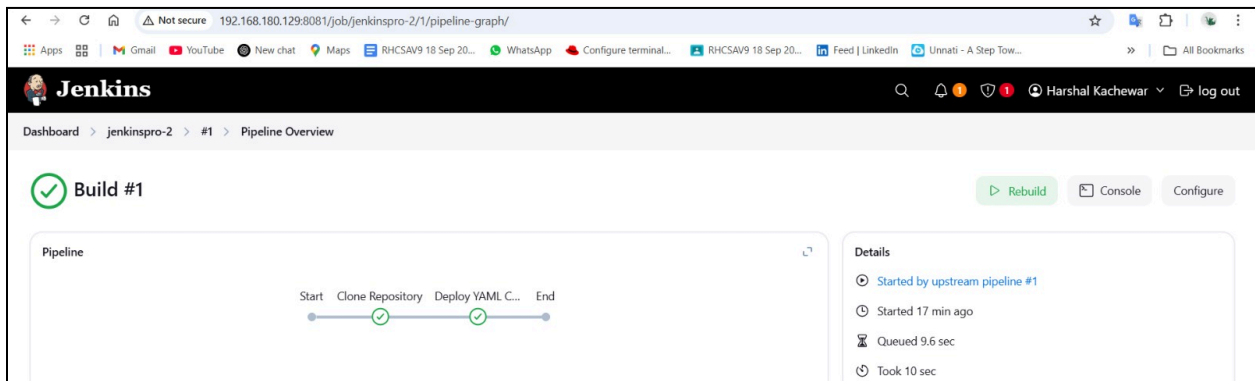
- Clone Repository
- Build Docker Image
- Tag Docker Image
- Push Docker Image
- Run Docker Container**

Stage 'Run Docker Container'

- Started 16 min ago
- Queued 0 ms
- Took 3.7 sec
- Success
- Running on Jenkins
- [View as plain text](#)

```

docker run -itd --name my-website -p 8086:80 harshalkachewar404/website:v1
Shell Script
0 + docker run -itd --name my-website -p 8086:80 harshalkachewar404/website:v1
1 42f437fd6290bda2aee9e4d8b557ea879ab9cd05fa0fcf35099feaa2f3f0292
  
```



Dashboard > jenkinspro-2 > #1 > Pipeline Overview

Build #1 Rebuild Console Configure

Pipeline

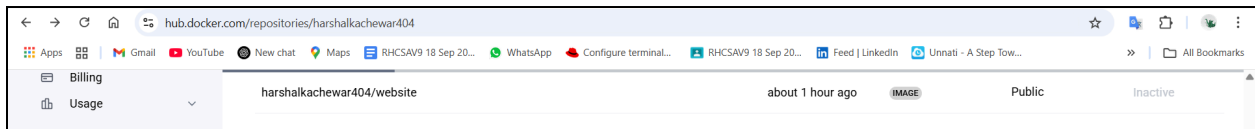
```

graph LR
    Start --> CloneRepository[Clone Repository]
    CloneRepository --> DeployYAML[Deploy YAML C...]
    DeployYAML --> End
  
```

Details

- Started by upstream pipeline #1
- Started 17 min ago
- Queued 9.6 sec
- Took 10 sec

Step 8: Verifying Image Push to Docker Hub

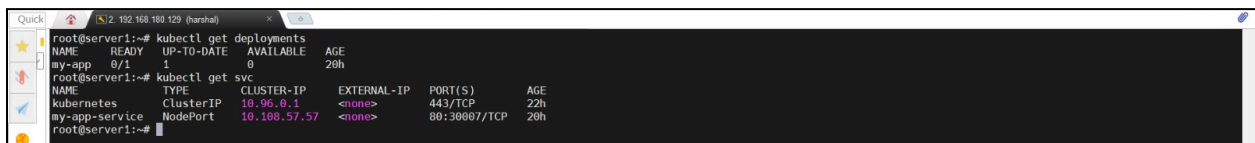


hub.docker.com/repositories/harshalkachewar404

harshalkachewar404/website

about 1 hour ago **IMAGE** Public Inactive

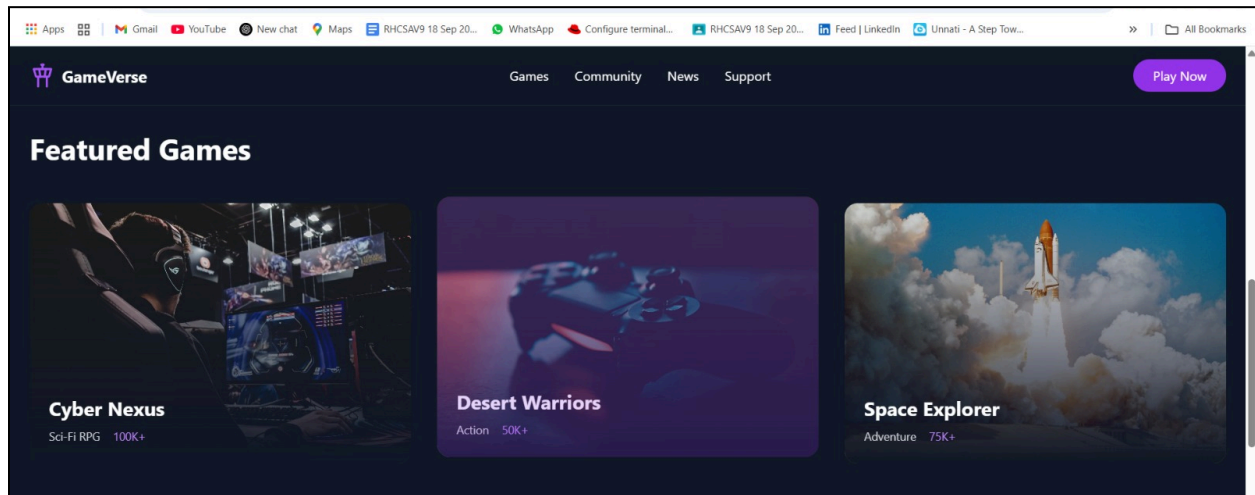
Step 9 : Verifying deployment and service created on server .



```

root@server1:~# kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
my-app    0/1     1             0           26h
root@server1:~# kubectl get svc
NAME      TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes  ClusterIP  10.96.0.1     <none>         443/TCP           22h
my-app-service  NodePort   10.100.57.57 <none>         80:30097/TCP      26h
root@server1:~#
  
```

Step 10: Verifying Deployment on Server



Conclusion

have successfully configured `jenkinspro-1` to build, tag, push, and run a Docker container, `jenkinspro-2` to deploy configurations after `jenkinspro-1` completes, and a triggering mechanism to automate sequential builds, ensuring that whenever `jenkinspro-1` runs, it automatically triggers `jenkinspro-2` for a smooth CI/CD pipeline workflow.